

ДИСЦИПЛИНА	Программирование в операционной системе Линукс
ИНСТИТУТ	Передовая инженерная школа СВЧ-электроники
КАФЕДРА	Передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания по дисциплине
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

Ссылка на материал:

<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-5>

Лекция №5: Инструментарий разработчика: компиляция и отладка

Теоретическая вводная

Компиляция в Linux: превращение кода в программу

Представьте, что вы написали письмо на английском языке, но ваш друг понимает только китайский. Компилятор — это профессиональный переводчик, который берет ваш код, написанный на языке, понятном человеку (C, C++, Rust), и превращает его в язык, который понимает процессор — машинный код. Но это не простой перевод слово за слово, а сложный многоэтапный процесс, где каждое преобразование подчиняется строгим правилам.

В мире Linux компилятор GCC (GNU Compiler Collection) — это настоящий швейцарский нож для разработчиков. Он не просто компилирует код, а проводит его через целую фабрику преобразований: препроцессор подготавливает код, удаляя комментарии и раскрывая макросы; компилятор превращает его в ассемблер; ассемблер создает объектные файлы; линковщик соединяет все части в единую программу. Это как собрать автомобиль из тысяч деталей — каждая должна быть на своем месте и правильно соединена с другими.

Этапы компиляции: за кулисами волшебства

Когда вы набираете `gcc main.c`, за этой простой командой скрывается четыре четких этапа. Препроцессор работает как умный текстовый редактор — он обрабатывает директивы, начинающиеся с решетки: подключает заголовочные файлы, раскрывает макросы, убирает комментарии. Результат его работы — чистый код, готовый к компиляции.

Затем компилятор берет этот подготовленный код и превращает его в ассемблер — низкоуровневый язык, близкий к машинному, но еще понятный человеку. Ассемблер уже специфичен для каждой архитектуры процессора, вот почему программа, скомпилированная для Intel, не будет работать на ARM.

Третий этап — ассемблирование. Ассемблер преобразует человеко-читаемый ассемблерный код в двоичный машинный код, создавая объектные файлы. Эти файлы содержат уже настоящие инструкции

для процессора, но они еще не являются законченной программой — в них есть дыры, места для подключения внешних функций.

Финальный этап — линковка. Линковщик берет все объектные файлы и соединяет их вместе, разрешая ссылки на внешние функции из библиотек. Он создает исполняемый файл, который операционная система может загрузить в память и выполнить.

Библиотеки: коллективный разум программирования

Библиотеки в Linux — это как набор стандартных деталей в конструкторе. Зачем каждый раз изобретать колесо, если можно взять готовое? Статические библиотеки (.a файлы) встраиваются прямо в вашу программу на этапе линковки — программа становится самодостаточной, но увеличивается в размере. Динамические библиотеки (.so файлы) остаются отдельными файлами и подгружаются в память при запуске программы — экономят место и память, но требуют, чтобы библиотека была доступна в системе.

Отладка: искусство находить невидимое

Отладка — это детективная работа, где вы ищете преступника (баг) по оставленным следам (симптомам). GDB (GNU Debugger) — ваш верный помощник в этом расследовании. Он позволяет заглянуть внутрь работающей программы, остановить ее в любой момент, посмотреть на значения переменных, проследить выполнение шаг за шагом.

Но отладка — это не только про поиск ошибок. Это способ глубоко понять, как работает ваша программа, как взаимодействуют ее части, как данные преобразуются в процессе выполнения. Хороший отладчик — это как рентгеновский аппарат для кода, позволяющий увидеть его внутреннюю структуру и поведение в реальном времени.

Флаги компиляции: настройка качества

Флаги компиляции — это не просто дополнительные опции, это тонкие регуляторы, которые позволяют настроить процесс сборки под ваши нужды. `-g` добавляет отладочную информацию, без которой GDB будет бесполезен. `-Wall` включает все предупреждения, заставляя компилятор быть более внимательным к потенциальным проблемам. `-O2` включает оптимизации, делая программу быстрее, но иногда усложняя отладку.

Понимание этих флагов — это как умение правильно настроить музыкальный инструмент перед концертом. Неправильная настройка — и красивая мелодия превращается в какофонию.

Сборка проектов: от хаоса к порядку

Когда проект растет и состоит из десятков файлов, компилировать каждый вручную становится невозможно. На помощь приходят системы сборки. Make — классический инструмент, который использует Makefile для описания правил сборки. Он определяет зависимости между файлами и пересобирает только то, что изменилось, экономя время разработчика.

Современные проекты часто используют более сложные системы вроде CMake или Autotools, но принцип остается тем же: автоматизировать рутинный процесс превращения исходного кода в готовую программу.

Практические примеры

Пример 1: Первая программа и базовые флаги компиляции

Цель: Научиться компилировать простую программу и понимать основные флаги GCC.

```
# 1. Создадим простую программу на C
cat > hello.c << 'EOF'
#include <stdio.h>

int main() {
    printf("Hello, Compilation World!\n");
    int x = 5;
    int y = 10;
    int sum = x + y;
    printf("Sum: %d\n", sum);
    return 0;
}
EOF

# 2. Базовая компиляция
gcc hello.c -o hello
./hello

# 3. Компиляция с отладочной информацией
gcc -g hello.c -o hello_debug
./hello_debug

# 4. Компиляция со всеми предупреждениями
gcc -Wall hello.c -o hello_wall
./hello_wall

# 5. Компиляция с оптимизацией
gcc -O2 hello.c -o hello_optimized
./hello_optimized

# 6. Посмотрим разницу в размере файлов
ls -la hello*

# 7. Компиляция с дополнительной информацией для отладки
gcc -g3 hello.c -o hello_debug_max

# 8. Компиляция с сохранением промежуточных файлов
gcc -save-temps hello.c -o hello_with_temps
ls -la hello*
```

Пример 2: Многофайловый проект и создание объектных файлов

Цель: Научиться работать с проектами из нескольких файлов.

```
# 1. Создадим заголовочный файл
cat > math_operations.h << 'EOF'
#ifndef MATH_OPERATIONS_H
#define MATH_OPERATIONS_H

int add(int a, int b);
int multiply(int a, int b);
double divide(double a, double b);

#endif
EOF

# 2. Создаем реализацию математических функций
cat > math_operations.c << 'EOF'
#include "math_operations.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}

double divide(double a, double b) {
    if (b == 0) {
        return 0; // Простая обработка ошибки
    }
    return a / b;
}
EOF

# 3. Создаем главный файл
cat > main.c << 'EOF'
#include <stdio.h>
#include "math_operations.h"

int main() {
    int result_add = add(5, 3);
    int result_mult = multiply(4, 7);
    double result_div = divide(10.0, 2.0);

    printf("Addition: %d\n", result_add);
    printf("Multiplication: %d\n", result_mult);
    printf("Division: %.2f\n", result_div);

    return 0;
}
EOF

# 4. Компилируем каждый файл в объектный файл отдельно
gcc -c math_operations.c -o math_operations.o
gcc -c main.c -o main.o
```

```
# 5. Линкуем объектные файлы в исполняемый файл
gcc main.o math_operations.o -o calculator

# 6. Запускаем программу
./calculator

# 7. Альтернативный способ: компиляция всех файлов сразу
gcc main.c math_operations.c -o calculator_direct
./calculator_direct
```

Пример 3: Создание и использование статических библиотек

Цель: Научиться создавать и использовать статические библиотеки.

```
# 1. Создаем объектные файлы для библиотеки
gcc -c math_operations.c -o math_operations.o

# 2. Создаем статическую библиотеку
ar rcs libmath.a math_operations.o

# 3. Проверяем содержимое библиотеки
ar t libmath.a
nm libmath.a

# 4. Компилируем главную программу с использованием библиотеки
gcc main.c -L. -lmath -o calculator_static

# 5. Запускаем программу
./calculator_static

# 6. Посмотрим, что библиотека встроена в исполняемый файл
ldd calculator_static

# 7. Создаем более сложную библиотеку с несколькими модулями
cat > string_operations.h << 'EOF'
#ifndef STRING_OPERATIONS_H
#define STRING_OPERATIONS_H

void reverse_string(char* str);
int string_length(const char* str);

#endif
EOF

cat > string_operations.c << 'EOF'
#include "string_operations.h"
#include <string.h>

void reverse_string(char* str) {
    int len = strlen(str);
```

```

    for (int i = 0; i < len / 2; i++) {
        char temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
}

```

```

int string_length(const char* str) {
    int count = 0;
    while (str[count] != '\0') {
        count++;
    }
    return count;
}

```

EOF

```

# 8. Создаем объектные файлы и добавляем их в библиотеку
gcc -c string_operations.c -o string_operations.o
ar rcs libutils.a math_operations.o string_operations.o

```

```

# 9. Используем объединенную библиотеку
gcc main.c -L. -lutils -o calculator_with_utils

```

Пример 4: Базовое использование GDB для отладки

Цель: Научиться основам работы с отладчиком GDB.

```

# 1. Создаем программу с преднамеренной ошибкой для отладки
cat > debug_me.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>

int calculate_factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * calculate_factorial(n - 1);
}

int main() {
    int number = 5;
    int result = calculate_factorial(number);
    printf("Factorial of %d is %d\n", number, result);

    // Преднамеренная ошибка - разыменование NULL указателя
    int* ptr = NULL;
    *ptr = 42; // Это вызовет segmentation fault

    return 0;
}
EOF

```

```
# 2. Компилируем с отладочной информацией
gcc -g debug_me.c -o debug_me

# 3. Запускаем программу под GDB
gdb ./debug_me

# Внутри GDB выполняем:
# run                - запускаем программу
# backtrace          - смотрим стек вызовов при падении
# quit              - выходим из GDB

# 4. Теперь найдем ошибку с помощью отладки
gdb ./debug_me

# Команды для выполнения внутри GDB:
# break main         - устанавливаем точку останова на main
# run                - запускаем программу
# next               - выполняем следующую строку
# print number       - печатаем значение переменной
# break calculate_factorial - точка останова на функции
# continue           - продолжаем выполнение до следующей точки останова
# step               - заходим внутрь функции
# where              - где мы находимся в коде
```

Пример 5: Продвинутая отладка с GDB

Цель: Освоить продвинутые техники отладки в GDB.

```
# 1. Создаем программу для сложной отладки
cat > advanced_debug.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>

int global_counter = 0;

void process_data(int* data, int size) {
    for (int i = 0; i <= size; i++) { // Ошибка: выход за границы массива
        data[i] = i * 2;
        global_counter++;
    }
}

void print_array(int* data, int size) {
    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");
}
```

```
int main() {
    int data[5];

    printf("Global counter: %d\n", global_counter);
    process_data(data, 5);
    printf("Global counter after processing: %d\n", global_counter);
    print_array(data, 5);

    return 0;
}
EOF
```

```
# 2. Компилируем с отладочной информацией
gcc -g advanced_debug.c -o advanced_debug
```

```
# 3. Запускаем отладку
gdb ./advanced_debug
```

```
# Команды для выполнения в GDB:
# list                - просмотр кода
# break process_data  - точка останова на функции
# run                 - запуск программы
# watch global_counter - отслеживание изменения переменной
# continue            - продолжение выполнения
# print data          - просмотр массива
# x/10w data          - просмотр памяти массива
# info breakpoints    - информация о точках останова
# delete breakpoint 1 - удаление точки останова 1
# set var global_counter=0 - изменение переменной во время выполнения
```

Пример 6: Работа с препроцессором

Цель: Понять работу препроцессора и научиться использовать макросы.

```
# 1. Создаем программу с макросами и условной компиляцией
cat > preprocessor_demo.c << 'EOF'
#include <stdio.h>

#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))
#define DEBUG 1

#ifdef DEBUG
    #define DBG_PRINT(x) printf("DEBUG: %s = %d\n", #x, x)
#else
    #define DBG_PRINT(x)
#endif

int main() {
    int x = 5, y = 10;
```



```
printf("Maximum: %d\n", MAX(x, y));
printf("Square of %d: %d\n", x, SQUARE(x));

DBG_PRINT(x);
DBG_PRINT(y);

// Демонстрация потенциальной проблемы с макросами
printf("Problematic macro: %d\n", SQUARE(x + 1)); // Ожидается 36, но будет 11

return 0;
}
EOF

# 2. Компилируем и смотрим результат препроцессора
gcc -E preprocessor_demo.c -o preprocessor_demo.i
head -50 preprocessor_demo.i

# 3. Компилируем обычным способом
gcc preprocessor_demo.c -o preprocessor_demo
./preprocessor_demo

# 4. Компилируем без DEBUG определения
gcc -DDEBUG=0 preprocessor_demo.c -o preprocessor_demo_no_debug
./preprocessor_demo_no_debug

# 5. Создаем заголовочный файл с защитой от повторного включения
cat > config.h << 'EOF'
#ifndef CONFIG_H
#define CONFIG_H

#define VERSION "1.0.0"
#define MAX_BUFFER_SIZE 1024

#endif
EOF

# 6. Проверяем работу защиты включения
cat > include_test.c << 'EOF'
#include "config.h"
#include "config.h" // Преднамеренное двойное включение

int main() {
    printf("Version: %s\n", VERSION);
    return 0;
}
EOF

gcc include_test.c -o include_test
./include_test
```

Пример 7: Оптимизации компилятора

Цель: Изучить влияние различных уровней оптимизации.

```
# 1. Создаем программу для тестирования оптимизаций
cat > optimization_test.c << 'EOF'
#include <stdio.h>
#include <time.h>

int expensive_calculation(int n) {
    int result = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            result += i * j;
        }
    }
    return result;
}

int main() {
    clock_t start = clock();

    int total = 0;
    for (int i = 0; i < 100; i++) {
        total += expensive_calculation(100);
    }

    clock_t end = clock();
    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result: %d\n", total);
    printf("Time: %.4f seconds\n", time_spent);

    return 0;
}
EOF

# 2. Компилируем без оптимизаций
gcc -O0 optimization_test.c -o optimization_test_00
time ./optimization_test_00

# 3. Компилируем с базовыми оптимизациями
gcc -O1 optimization_test.c -o optimization_test_01
time ./optimization_test_01

# 4. Компилируем с агрессивными оптимизациями
gcc -O2 optimization_test.c -o optimization_test_02
time ./optimization_test_02

# 5. Компилируем с максимальными оптимизациями
gcc -O3 optimization_test.c -o optimization_test_03
time ./optimization_test_03

# 6. Сравним размеры исполняемых файлов
ls -la optimization_test_*
```

```
# 7. Посмотрим на разницу в ассемблерном коде
gcc -O0 -S optimization_test.c -o optimization_test_00.s
gcc -O2 -S optimization_test.c -o optimization_test_02.s

# 8. Сравним количество строк в ассемблерных файлах
wc -l optimization_test_*.s
```

Пример 8: Работа с профилированием gprof

Цель: Научиться использовать профилировщик для оптимизации производительности.

```
# 1. Создаем программу для профилирования
cat > profile_me.c << 'EOF'
#include <stdio.h>
#include <time.h>

void fast_function() {
    // Быстрая функция
    for (int i = 0; i < 1000; i++) {
        volatile int x = i * 2; // volatile чтобы компилятор не оптимизировал
    }
}

void slow_function() {
    // Медленная функция
    for (int i = 0; i < 1000000; i++) {
        volatile int x = i * 3;
    }
}

void medium_function() {
    // Функция средней скорости
    for (int i = 0; i < 100000; i++) {
        volatile int x = i * 4;
    }
}

int main() {
    printf("Starting profiling demo...\n");

    for (int i = 0; i < 10; i++) {
        fast_function();
        medium_function();
        slow_function();
    }

    printf("Profiling demo completed.\n");
    return 0;
}
EOF
```

```
# 2. Компилируем с поддержкой профилирования
gcc -pg profile_me.c -o profile_me

# 3. Запускаем программу (создаст файл gmon.out)
./profile_me

# 4. Анализируем результаты профилирования
gprof profile_me gmon.out > analysis.txt

# 5. Смотрим результаты
head -30 analysis.txt

# 6. Создаем визуализацию (если установлен gprof2dot)
# gprof profile_me | gprof2dot | dot -Tpng -o profile.png

# 7. Альтернативный способ: текстовый анализ
cat analysis.txt | grep -A 10 "time seconds"

# 8. Очищаем временные файлы
rm -f gmon.out analysis.txt
```

Пример 9: Статический анализ кода

Цель: Научиться использовать инструменты статического анализа.

```
# 1. Устанавливаем инструменты статического анализа
sudo apt install splint cppcheck

# 2. Создаем программу с потенциальными проблемами
cat > static_analysis.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void potential_problems(char* input) {
    char buffer[10];
    strcpy(buffer, input); // Потенциальное переполнение буфера

    int uninitialized;
    if (uninitialized > 0) { // Использование неинициализированной переменной
        printf("Positive\n");
    }

    int* ptr = malloc(sizeof(int));
    // Утечка памяти - нет free(ptr)
}

int main() {
    char test[] = "This is a very long string that might cause problems";
    potential_problems(test);
}
```

```
    return 0;
}
EOF

# 3. Используем splint для статического анализа
splint static_analysis.c

# 4. Используем cppcheck
cppcheck --enable=all static_analysis.c

# 5. Компилируем с дополнительными проверками GCC
gcc -Wall -Wextra -Wpedantic static_analysis.c -o static_analysis

# 6. Используем санитайзеры для динамического анализа
gcc -fsanitize=address -fsanitize=undefined static_analysis.c -o
static_analysis_sanitized
./static_analysis_sanitized

# 7. Анализируем с помощью valgrind (если установлен)
# valgrind --leak-check=full ./static_analysis

# 8. Создаем более качественный код и проверяем снова
cat > fixed_code.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void safe_function(const char* input) {
    char buffer[10];
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\\0';

    int initialized = 0;
    if (initialized > 0) {
        printf("Positive\\n");
    }

    int* ptr = malloc(sizeof(int));
    if (ptr != NULL) {
        *ptr = 42;
        free(ptr);
    }
}

int main() {
    char test[] = "Short";
    safe_function(test);
    return 0;
}
EOF

# 9. Проверяем исправленный код
splint fixed_code.c
cppcheck fixed_code.c
```

Пример 10: Создание и использование shared библиотек

Цель: Научиться работать с динамическими библиотеками.

```
# 1. Создаем shared library
gcc -c -fPIC math_operations.c -o math_operations_pic.o
gcc -shared -o libmath.so math_operations_pic.o

# 2. Компилируем программу с dynamic linking
gcc main.c -L. -lmath -o calculator_dynamic

# 3. Пробуем запустить (скорее всего не найдет библиотеку)
./calculator_dynamic || echo "Library not found"

# 4. Добавляем текущую директорию в путь поиска библиотек
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
./calculator_dynamic

# 5. Проверяем зависимости
ldd calculator_dynamic

# 6. Устанавливаем библиотеку в системную директорию (для демонстрации)
sudo cp libmath.so /usr/local/lib/
sudo ldconfig

# 7. Теперь можем запускать без LD_LIBRARY_PATH
unset LD_LIBRARY_PATH
./calculator_dynamic

# 8. Создаем версию библиотеки
gcc -shared -Wl,-soname,libmath.so.1 -o libmath.so.1.0 math_operations_pic.o
ln -sf libmath.so.1.0 libmath.so.1
ln -sf libmath.so.1 libmath.so

# 9. Проверяем симлинки
ls -la libmath.so*

# 10. Очищаем системную директорию
sudo rm /usr/local/lib/libmath.so
sudo ldconfig
```

Пример 11: Работа с системой сборки Make

Цель: Научиться создавать и использовать Makefiles.

```
# 1. Создаем простой Makefile
cat > Makefile << 'EOF'
# Компилятор и флаги
```

```
CC = gcc
CFLAGS = -Wall -Wextra -g
LDFLAGS =

# Цели
TARGET = calculator
SOURCES = main.c math_operations.c
OBJECTS = $(SOURCES:.c=.o)

# Правила по умолчанию
all: $(TARGET)

$(TARGET): $(OBJECTS)
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(TARGET) $(OBJECTS)

install: $(TARGET)
    cp $(TARGET) /usr/local/bin/

uninstall:
    rm -f /usr/local/bin/$(TARGET)

.PHONY: all clean install uninstall
EOF

# 2. Собираем проект с помощью make
make

# 3. Запускаем программу
./calculator

# 4. Очищаем собранные файлы
make clean

# 5. Собираем снова
make

# 6. Создаем более сложный Makefile с зависимостями
cat > Makefile.advanced << 'EOF'
CC = gcc
CFLAGS = -Wall -Wextra -g -MD
LDFLAGS =
TARGET = calculator
SOURCES = main.c math_operations.c string_operations.c
OBJECTS = $(SOURCES:.c=.o)
DEPS = $(OBJECTS:.o=.d)

all: $(TARGET)
```

```
$(TARGET): $(OBJECTS)
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(TARGET) $(OBJECTS) $(DEPS)

distclean: clean
    rm -f *~

install: $(TARGET)
    cp $(TARGET) /usr/local/bin/

# Включаем зависимости
-include $(DEPS)

.PHONY: all clean distclean install
EOF

# 7. Используем продвинутый Makefile
make -f Makefile.advanced

# 8. Смотрим созданные файлы зависимостей
cat main.d
```

Пример 12: Отладка с помощью Valgrind

Цель: Научиться использовать Valgrind для поиска утечек памяти и ошибок.

```
# 1. Устанавливаем Valgrind
sudo apt install valgrind

# 2. Создаем программу с утечками памяти
cat > memory_leak.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void create_leak() {
    char* buffer = malloc(100);
    strcpy(buffer, "This memory will be leaked");
    // Нет free(buffer) - утечка памяти
}

void double_free() {
    int* ptr = malloc(sizeof(int));
    *ptr = 42;
    free(ptr);
    free(ptr); // Двойное освобождение
```



```
}

void use_after_free() {
    char* str = malloc(50);
    strcpy(str, "Hello");
    free(str);
    printf("%s\n", str); // Использование после освобождения
}

int main() {
    printf("Memory leak demo\n");
    create_leak();

    // double_free(); // Раскомментировать для демонстрации
    // use_after_free(); // Раскомментировать для демонстрации

    return 0;
}
EOF

# 3. Компилируем с отладочной информацией
gcc -g memory_leak.c -o memory_leak

# 4. Запускаем Valgrind для проверки утечек
valgrind --leak-check=full ./memory_leak

# 5. Анализируем вывод Valgrind
# Обращаем внимание на:
# - definitely lost: точно утерянная память
# - indirectly lost: косвенно утерянная
# - possibly lost: возможно утерянная

# 6. Запускаем с дополнительными проверками
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all ./memory_leak

# 7. Проверяем кеш промахи (если нужно)
# valgrind --tool=cachegrind ./memory_leak

# 8. Создаем исправленную версию
cat > memory_fixed.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void no_leak() {
    char* buffer = malloc(100);
    if (buffer != NULL) {
        strcpy(buffer, "This memory will be properly freed");
        printf("%s\n", buffer);
        free(buffer);
    }
}

void proper_memory_management() {
```

```
int* ptr = malloc(sizeof(int));
if (ptr != NULL) {
    *ptr = 42;
    free(ptr);
    ptr = NULL; // Хорошая практика
}

int main() {
    printf("Proper memory management demo\n");
    no_leak();
    proper_memory_management();
    return 0;
}
EOF
```

```
# 9. Проверяем исправленную версию
gcc -g memory_fixed.c -o memory_fixed
valgrind --leak-check=full ./memory_fixed
```

Пример 13: Кросс-компиляция и разные архитектуры

Цель: Понять основы кросс-компиляции.

```
# 1. Проверяем текущую архитектуру
uname -m

# 2. Компилируем с указанием архитектуры
gcc -m32 hello.c -o hello_32bit 2>/dev/null || echo "32-bit support not installed"

# 3. Устанавливаем поддержку 32-битных программ (если нужно)
sudo apt install gcc-multilib

# 4. Пробуем снова
gcc -m32 hello.c -o hello_32bit
file hello_32bit

# 5. Сравниваем с 64-битной версией
gcc hello.c -o hello_64bit
file hello_64bit

# 6. Смотрим различия в размерах
ls -la hello_*

# 7. Компилируем с разными уровнями оптимизации для разных архитектур
gcc -m32 -O2 hello.c -o hello_32bit_O2
gcc -m64 -O2 hello.c -o hello_64bit_O2

# 8. Создаем программу для тестирования выравнивания памяти
cat > alignment_test.c << 'EOF'
#include <stdio.h>
```

```
#include <stddef.h>

struct test_struct {
    char a;
    int b;
    char c;
    double d;
};

int main() {
    printf("Size of struct: %zu\n", sizeof(struct test_struct));
    printf("Offset of a: %zu\n", offsetof(struct test_struct, a));
    printf("Offset of b: %zu\n", offsetof(struct test_struct, b));
    printf("Offset of c: %zu\n", offsetof(struct test_struct, c));
    printf("Offset of d: %zu\n", offsetof(struct test_struct, d));
    return 0;
}
EOF

# 9. Компилируем для разных архитектур и сравниваем
gcc -m32 alignment_test.c -o alignment_32
gcc -m64 alignment_test.c -o alignment_64

./alignment_32
./alignment_64
```

Пример 14: Работа с ассемблером

Цель: Понять связь между C кодом и ассемблером.

```
# 1. Компилируем C код в ассемблер
gcc -S hello.c -o hello.s

# 2. Смотрим сгенерированный ассемблерный код
cat hello.s

# 3. Компилируем с разными уровнями оптимизации и сравниваем
gcc -S -O0 hello.c -o hello_00.s
gcc -S -O2 hello.c -o hello_02.s

# 4. Сравниваем размеры ассемблерных файлов
wc -l hello_*.s

# 5. Создаем программу с inline ассемблером
cat > inline_asm.c << 'EOF'
#include <stdio.h>

int main() {
    int a = 5, b = 10, result;

    // Inline assembly для сложения
```

```

    asm volatile (
        "add %1, %2, %0"
        : "=r" (result)
        : "r" (a), "r" (b)
    );

    printf("Result: %d\n", result);

    // Получаем значение регистра
    unsigned long stack_pointer;
    asm volatile ("mov %0, sp" : "=r" (stack_pointer));
    printf("Stack pointer: 0x%lx\n", stack_pointer);

    return 0;
}
EOF

```

```

# 6. Компилируем и запускаем
gcc inline_asm.c -o inline_asm
./inline_asm

```

```

# 7. Смотрим ассемблерный код с inline ассемблером
gcc -S inline_asm.c -o inline_asm.s
cat inline_asm.s

```

```

# 8. Создаем чистый ассемблерный файл
cat > pure_asm.s << 'EOF'
.global _start
.section .text

```

```

_start:
    # write system call
    mov x0, #1          # stdout
    ldr x1, =message    # buffer
    ldr x2, =len         # length
    mov x8, #64         # write syscall number
    svc #0              # system call

    # exit system call
    mov x0, #0          # exit status
    mov x8, #93         # exit syscall number
    svc #0

.section .data
message:
    .asciz "Hello from pure assembly!\n"
len = . - message
EOF

```

```

# 9. Компилируем и линкуем ассемблерную программу
# as pure_asm.s -o pure_asm.o
# ld pure_asm.o -o pure_asm
# ./pure_asm

```

Пример 15: Интеграция с IDE и системами сборки

Цель: Научиться настраивать продвинутые системы сборки.

```
# 1. Создаем CMakeLists.txt для CMake
cat > CMakeLists.txt << 'EOF'
cmake_minimum_required(VERSION 3.10)
project(CalculatorProject)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)

# Исполняемый файл
add_executable(calculator
    main.c
    math_operations.c
    string_operations.c
)

# Настройки компилятора
target_compile_options(calculator PRIVATE -Wall -Wextra -g)

# Статическая библиотека
add_library(math_static STATIC math_operations.c)
target_include_directories(math_static PUBLIC .)

# Shared библиотека
add_library(math_shared SHARED math_operations.c)
target_include_directories(math_shared PUBLIC .)

# Тесты (если есть)
enable_testing()
add_test(NAME calculator_test COMMAND calculator)
EOF

# 2. Создаем build директорию и собираем проект
mkdir build
cd build
cmake ..
make

# 3. Запускаем программу
./calculator

# 4. Смотрим какие цели доступны
make help

# 5. Создаем простой конфигурационный файл для autotools
cat > configure.ac << 'EOF'
AC_INIT([calculator], [1.0], [your@email.com])
AM_INIT_AUTOMAKE
AC_PROG_CC
```

```
AC_CONFIG_FILES([Makefile])
AC_OUTPUT
EOF

cat > Makefile.am << 'EOF'
bin_PROGRAMS = calculator
calculator_SOURCES = main.c math_operations.c string_operations.c
EOF

# 6. Устанавливаем pkg-config файл для нашей библиотеки
mkdir -p pkgconfig
cat > pkgconfig/libmath.pc << 'EOF'
prefix=/usr/local
exec_prefix=${prefix}
libdir=${exec_prefix}/lib
includedir=${prefix}/include

Name: libmath
Description: Simple math library
Version: 1.0.0
Libs: -L${libdir} -lmath
Cflags: -I${includedir}
EOF

# 7. Создаем скрипт для сборки разных конфигураций
cat > build_all.sh << 'EOF'
#!/bin/bash

echo "Building debug version..."
mkdir -p build_debug
cd build_debug
cmake -DCMAKE_BUILD_TYPE=Debug ..
make
cd ..

echo "Building release version..."
mkdir -p build_release
cd build_release
cmake -DCMAKE_BUILD_TYPE=Release ..
make
cd ..

echo "Building with sanitizers..."
mkdir -p build_sanitize
cd build_sanitize
cmake -DCMAKE_BUILD_TYPE=Debug -DUSE_SANITIZERS=ON ..
make
cd ..

echo "Build complete!"
ls -la build_*/calculator
EOF
```

```
chmod +x build_all.sh  
./build_all.sh
```